

Wavelets în Procesarea Imaginilor

De la Fourier la JPEG2000

Suport scris pentru prezentarea interactivă

Neamțu Alexandra

Live: <https://ale0204.student-dev.ro/dsp/>

Curs: Prelucrarea Semnalelor

CTI, anul III, semestrul 1

Titular de curs: prof. univ. dr. habil. Paul Irofti

Facultatea de Matematică și Informatică,
Universitatea din București

Data: 28 august 2026

Cuprins

1	Despre acest suport	4
1.1	Harta: de la ecran la capitol	4
2	Fourier, filtre și convoluție: instrumentele de bază	7
2.1	Transformata Fourier	7
2.2	Filtre digitale	8
2.3	Convoluția	10
2.4	Fourier în două dimensiuni	10
2.5	Kernele 2D, pe familii	11
3	Transformata wavelet și algoritmul lui Mallat	13
3.1	De ce wavelets	13
3.2	Familii de wavelets	14
3.3	Wavelet-ul complex și faza	14
3.4	Algoritmul lui Mallat	15
4	Compresia imaginilor: JPEG versus JPEG2000	18
4.1	Pipeline-ul JPEG	18
4.2	O comparație la bitrate egal	19
5	Denoising prin prag	22
6	Aplicație: electrocardiograma	24
6.1	De ce wavelets, aici	24
6.2	Demo	24
7	Aplicație: EEG	26
8	Aplicație: recunoașterea muzicii	27
8.1	Cum funcționează amprenta	27
8.2	Demo	27
9	Sinteză: un singur mecanism, multe aplicații	29
10	Cum e construit, verificat și publicat	31
10.1	Pe scurt, arhitectura	31
10.2	Verificare automată	31
10.3	Rulare locală	33
10.4	Publicarea pe server	33
	Bibliografie	34

Listă de figuri

1.1	Ecranul de deschidere al turului: cârligul leagă subiectul de o cerință de specificație reală. Cinematograful digital impune JPEG2000 în locul lui JPEG, prin specificația DCI [1].	5
1.2	Cuprinsul turului: nouă secțiuni, cu carduri clickabile care sar direct la secțiunea aleasă.	5
2.1	Domeniul timp și spectrul de magnitudine calculate live pentru presetul „Două frecvențe”: vârfurile spectrale cad exact la 5 și 12 Hz, frecvențele din formula afișată dedesubt.	8
2.2	Cele trei forme ale filtrului trece-jos, toate normalizate la -3 dB pe frecvența de tăiere f_c	9
2.3	Filtru trece-jos Butterworth de ordinul 4: răspunsul în frecvență (stânga sus), spectrul semnalului înainte și după filtrare (dreapta sus) și semnalul filtrat în timp (jos), toate recalculate la orice schimbare de parametru.	9
2.4	Convoluția 2D cu semnul corect și nota despre corelație versus convoluție, alături de trei kernele reprezentative: blur Gaussian, sharpen și Sobel X (asimetric).	10
2.5	Aceeași imagine în cele două domenii: originalul în nivele de gri (stânga) și spectrul lui de magnitudine 2D pe scară logaritmică (dreapta), cu frecvențele joase în centru. Blana deasă a babuinului umple spectrul spre margini, semnătura unei texturi fine.	11
2.6	Blur Gaussian 3×3 aplicat pe imaginea Peppers, cu matricea de kernel afișată exact așa cum e trimisă către back-end.	12
3.1	Compromisul timp-frecvență, teoretic (stânga) și pe un semnal concret (dreapta).	13
3.2	Cele patru rezultate din spatele familiilor wavelet: media nulă, ecuațiile de scalare/wavelet, perechea CQF și non-redundanța DWT, fiecare cu citarea lui.	14
3.3	Wavelet-ul Morlet complex ca spirală: partea reală proiectată pe un perete, partea imaginară pe celălalt, iar culoarea curbei centrale codifică faza. Frecvența și unghiul de rotație sunt reglabile din dreapta.	15
3.4	Un nivel al algoritmului lui Mallat pas cu pas: fereastra filtrului, produsul, suma și decimarea cu doi, pe un semnal treaptă de 16 eșantioane.	16
3.5	Multi-nivel (stânga) și reconstrucția care închide bucla (dreapta).	17
4.1	Prima etapă a pipeline-ului JPEG pe o fotografie reală: separarea Y/Cb/Cr, cu formulele de conversie folosite efectiv de back-end.	18
4.2	Comparație la bitrate egal (≈ 1.01 bpp): wavelet câștigă pe PSNR și pierde la limită pe SSIM, cu ambele metrice afișate pe ecran.	21
5.1	Thresholding moale versus dur și distribuția tipică a coeficienților, cu pragul universal Donoho-Johnstone alături de pragul 2σ folosit efectiv de demo.	22
5.2	Denoising pe o imagine reală: reziduul amplificat $\times 10$ e zgomot granular pur, fără nicio structură a imaginii, semn că a rămas doar zgomotul.	23
6.1	ECG sintetic cu zgomot muscular: denoising wavelet și detecția celor șase băți, SNR de la -2.9 la 2.1 dB.	25
7.1	Banda alfa (8-13 Hz) extrasă cu un filtru trece-bandă dintr-un EEG sintetic. Nota din josul figurii explică de ce nu DWT.	26

8.1	Trei acorduri interferând în domeniul timp. Textul explică amprente ($f_1, f_2, \Delta t$) pe care le caută algoritmul real.	28
9.1	Ecranul de sinteză al turului: descompune / taie / reconstruiește, aceleași trei verbe explicând ECG, EEG, recunoașterea muzicii, denoising și JPEG2000.	30
10.1	Un ecran tipic pentru trecerea de interacțiune: selector de sprite, selector de kernel, cursor de dimensiune, selector de margine, cursor de viteză și cinci butoane de transport, fiecare apăsător sau tras automat, cu o captură înainte și una după.	32

Capitolul 1

Despre acest suport

Fiecare ecran al prezentării calculează pe loc ce arată. Un front-end React randează cincizeci de ecrane, iar un back-end FastAPI rulează transformata cerută, la fiecare interacțiune, pe semnalul sau imaginea aleasă de cel care o urmărește, ori de câte ori pe ecran apare o cifră măsurată: un demo de filtrare aplică un filtru real pe un semnal real, un demo de compresie rulează o transformată de cosinus discretă și o transformată wavelet discretă adevărate, pe imagini standard de test. Rezultatul seamănă mai mult cu un instrument de laborator decât cu un set de diapozitive: un cursor care schimbă ordinul unui filtru Butterworth și redesenează răspunsul în frecvență pe loc transmite mai mult decât un paragraf despre ce înseamnă „ordinul controlează abrupțitatea”.

Acest document e gândit pentru cine urmărește prezentarea, live sau din înregistrare, și vrea ceva scris alături, la care să revină. Aceleași formule, dar derivate. Aceleași cifre, dar cu metoda din spatele lor. Iar la electrocardiogramă și la recunoașterea muzicii, mai mult decât arată cele câteva minute de pe scenă. Capitolele nu urmează neapărat ordinea ecranelor. Sunt grupate pe teme, ca fiecare aplicație practică (compresia imaginilor, eliminarea zgomotului, ECG, EEG, recunoașterea audio) să se poată citi de sine stătător, cu teoria strict necesară adusă înainte, nu presărată pe parcurs.

Prezentarea în sine e un tur ghidat de cincizeci de ecrane, parcurse cu săgețile tastaturii, grupate în nouă secțiuni vizibile într-o bară laterală (Figura 1.2). Fiecare ecran are propriul *deep link* și poate fi deschis direct, iar secțiunile se pot explora și individual, în afara turului. Live, la <https://ale0204.student-dev.ro/dsp/>.

1.1 Harta: de la ecran la capitol

Prezentarea are cincizeci de ecrane, numerotate în bara de jos a aplicației. Tabelul 1.1 spune, pentru fiecare grup de ecrane, unde se citește aceeași materie în textul de față. Citit invers, spune ce ecran să deschizi pentru un capitol anume.

Tabela 1.1: Corespondența dintre ecranele turului și capitolele acestui text.

Ecrane	Secțiunea din tur	Unde se citește
1–2	Deschidere, cuprins	Capitolul 1
3–5	Transformata Fourier	Capitolul 2
6–9	Filtre digitale	Secțiunea 2.2
10–13	Convoluția	Capitolul 2
14–16	Kernel-uri 2D	Secțiunea 2.5
17–24	Transformata wavelet	Capitolul 3
25–32	Algoritmul lui Mallat	Secțiunea 3.4
33–35	Aplicații, ECG	Capitolul 6
36–37	EEG	Capitolul 7
38–39	Amprenta audio	Capitolul 8
40–42	Denoising	Capitolul 5
43–47	DCT față de wavelet	Capitolul 4
48–50	Sinteză, bibliografie	Capitolul 9



Figura 1.1: Ecranul de deschidere al turului: cârligul leagă subiectul de o cerință de specificație reală. Cinematograful digital impune JPEG2000 în locul lui JPEG, prin specificația DCI [1].



Figura 1.2: Cuprinsul turului: nouă secțiuni, cu carduri clickabile care sar direct la secțiunea aleasă.

Capitolul 10 nu are ecrane proprii: descrie cum este construită și verificată aplicația.

Capitolul 2

Fourier, filtre și convoluție: instrumentele de bază

Restul documentului se sprijină pe trei idei simple, în ordine: orice semnal e o sumă de frecvențe, un filtru alege care dintre ele rămân, iar convoluția e mecanismul care face asta. Wavelets, câteva capitole mai încolo, e ce se întâmplă când combini toate trei cu o singură idee nouă.

2.1 Transformata Fourier

Shazam recunoaște o piesă din zece secunde de audio pentru că amprenta ei de frecvențe e unică [2], adică exact amprenta pe care o calculează transformata Fourier (revenim pe larg la Shazam în Capitolul 8). Rezultatul central al transformatei e că orice semnal se scrie ca o sumă, sau pentru semnale continue o integrală, de sinusoidale [3]:

$$F(\omega) = \int_{-\infty}^{\infty} f(t) e^{-i\omega t} dt, \quad f(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} F(\omega) e^{i\omega t} d\omega,$$

unde pulsația $\omega = 2\pi f$ e aceeași mărime ca frecvența f , doar în alte unități (rad/s față de Hz). Formula lui Euler, $e^{i\theta} = \cos \theta + i \sin \theta$, e ce leagă exponențiala complexă de sinusoidale. Teorema lui Parseval,

$$\int_{-\infty}^{\infty} |f(t)|^2 dt = \frac{1}{2\pi} \int_{-\infty}^{\infty} |F(\omega)|^2 d\omega,$$

spune că energia semnalului e aceeași, calculată în timp sau în frecvență. E o proprietate la care revenim când arătăm că nici transformata wavelet discretă nu pierde energie.

Limita transformatei contează la fel de mult ca rezultatul: Fourier spune CE frecvențe conține un semnal, dar nu CÂND apar ele. Un mashup din două piese are, pe porțiuni lungi, un spectru foarte asemănător cu suprapunerea lor, tocmai de-aceia Shazam rulează transformata pe ferestre scurte și succesive, nu pe piesa întreagă (transformata Fourier pe termen scurt, STFT). Fereastra scurtă fixează unde se pierde: cu cât fereastra e mai scurtă, cu atât localizarea în timp e mai bună și rezoluția în frecvență mai slabă, exact compromisul pe care wavelets îl rezolvă altfel, cu ferestre de dimensiune variabilă (Capitolul 3).

Demo-ul din tur (Figura 2.1) calculează efectiv o transformată Fourier rapidă (`np.fft.fft`) pentru opt funcții predefinite: sinusoidă simplă, două și trei frecvențe, chirp, puls Gaussian, undă pătrată, suma Fourier a unei pătrate și o expresie personalizată, evaluată printr-o listă restrânsă de funcții permise (`sin`, `cos`, `exp`, `pi`, `abs`, `sqrt`). Afășează simultan domeniul timp și spectrul de magnitudine.

Presetul cu suma Fourier a unei pătrate arată o limită care revine mai târziu. Cu fiecare armonică adăugată suma se apropie de unda pătrată, dar lângă discontinuitate rămâne o supraoscilație de aproape nouă la sută din saltul semnalului. Ea se îngustează pe măsură ce adaugi termeni și nu scade niciodată în înălțime. Fenomenul poartă numele lui Gibbs și e ce se întâmplă ori de câte ori tai brusc un spectru. Pentru presetul din figură, $f(t) = \sin(2\pi \cdot 5t) + \sin(2\pi \cdot 12t)$, cele două vârfuri spectrale cad exact la 5 și 12 Hz.

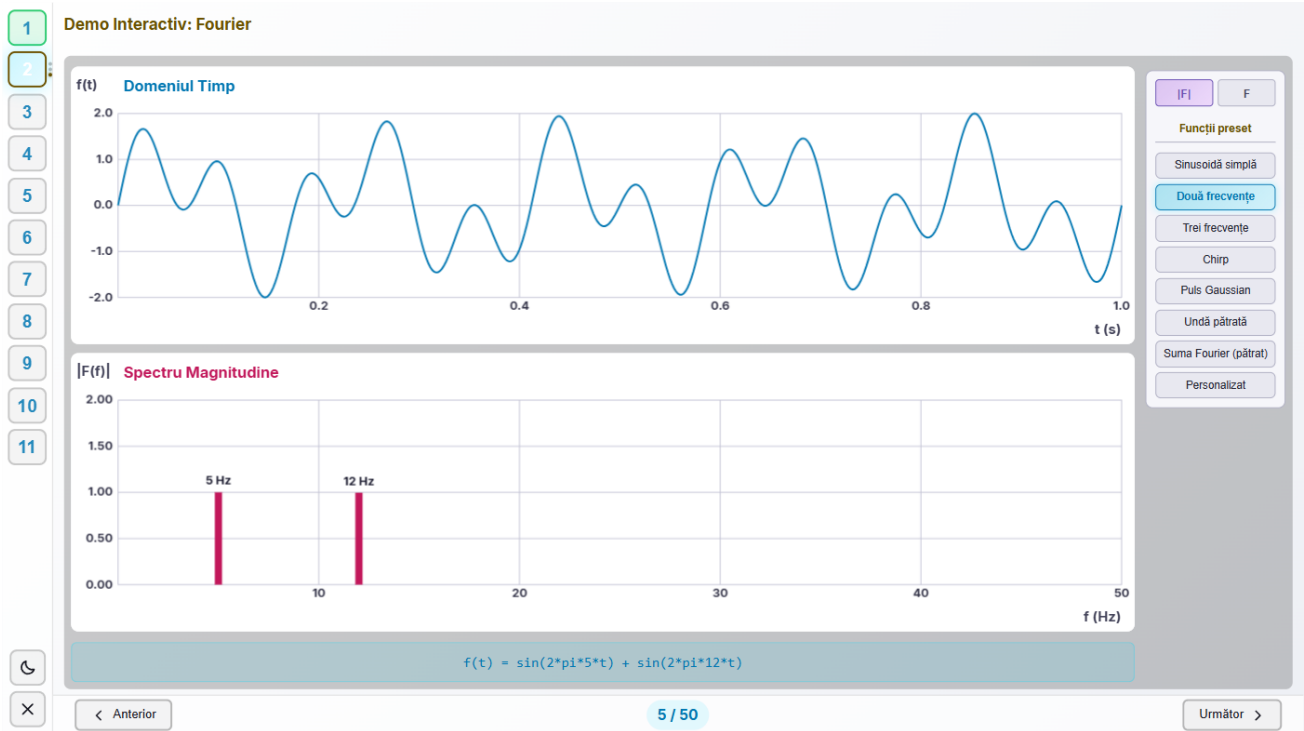


Figura 2.1: Domeniul timp și spectrul de magnitudine calculate live pentru presetul „Două frecvențe”: vârfurile spectrale cad exact la 5 și 12 Hz, frecvențele din formula afișată dedesubt.

2.2 Filtre digitale

Un filtru păstrează o parte din spectru și atenuează restul, teoria standard de proiectare a filtrelor digitale [19]. Trei forme apar în tur, toate normalizate să treacă prin -3 dB (adică $|H| = 1/\sqrt{2}$, jumătate din putere) exact la frecvența de tăiere f_c , ca să fie comparabile la aceeași tăietură:

$$H_{LP}^{\text{ideal}}(f) = \begin{cases} 1 & |f| \leq f_c \\ 0 & |f| > f_c \end{cases}$$

$$|H_{LP}^{\text{Butterworth}}(f)|^2 = \frac{1}{1 + (f/f_c)^{2n}}$$

$$H_{LP}^{\text{Gaussian}}(f) = e^{-\frac{\ln 2}{2}(f/f_c)^2}$$

Filtrul ideal are un prag dur chiar la f_c , imposibil de realizat în practică: răspunsul lui în timp e un *sinc* infinit, deci necauzal, iar trunchierea lui readuce oscilația Gibbs întâlnită deja la Fourier. Butterworth și Gaussian sunt calibrate să dea exact $|H|^2 = \frac{1}{2}$ la $f = f_c$, oricare ar fi ordinul n sau lățimea Gaussianei. Ordinul controlează doar cât de abrupt coboară restul curbei, de la $n = 1$ (lent) la $n = 8$ (aproape ideal). Varianta trece-sus e complementul în putere al variantei trece-jos,

$$H_{HP}(f) = \sqrt{1 - |H_{LP}(f)|^2}, \quad \text{deci} \quad |H_{LP}|^2 + |H_{HP}|^2 = 1$$

la orice frecvență: cele două filtre împart mereu toată puterea, indiferent de formă sau ordin. Varianta trece-bandă înlanțuie un trece-sus la tăietura joasă cu un trece-jos la tăietura înaltă, $H_{BP}(f) = \sqrt{1 - |H_{LP}(f; f_{lo})|^2} \cdot H_{LP}(f; f_{hi})$.

În back-end, o singură funcție, `filter_magnitude`, produce atât curba de răspuns desenată pe ecran (Figura 2.3), cât și multiplicatorul aplicat efectiv semnalului: cererea `/api/filters/apply-signal` transformă semnalul cu FFT, îl înmulțește cu acel multiplicator și aplică FFT inversă. Multiplicatorul e real și par în frecvență, așa că semnalul filtrat rămâne real și filtrul e zero-fază. Iar faptul că e o singură funcție înseamnă că demo-ul nu poate afișa un filtru și calcula altul, o clasă întreagă de bug posibil eliminată prin construcție.



Figura 2.2: Cele trei forme ale filtrului trece-jos, toate normalizate la -3 dB pe frecvența de tăiere f_c .

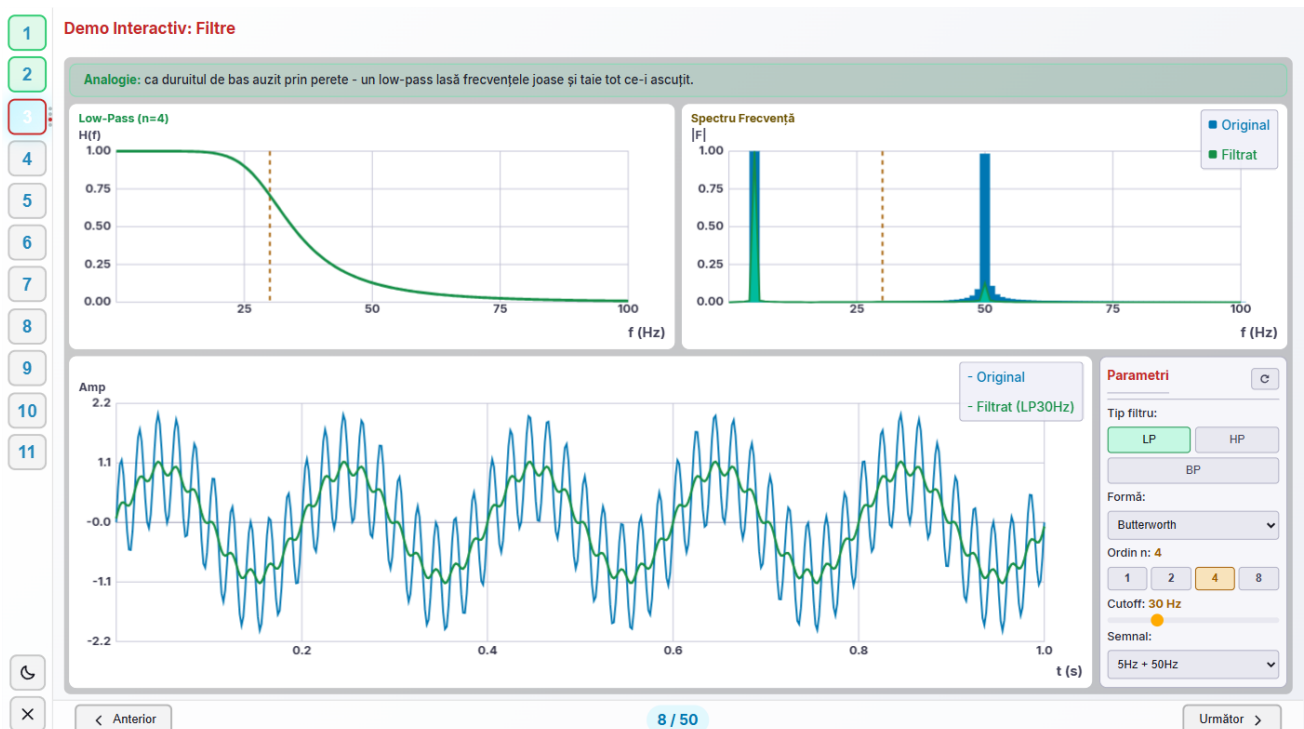


Figura 2.3: Filtru trece-jos Butterworth de ordinul 4: răspunsul în frecvență (stânga sus), spectrul semnalului înainte și după filtrare (dreapta sus) și semnalul filtrat în timp (jos), toate recalculate la orice schimbare de parametru.

Băncile de filtre, cu un trece-jos h pentru coeficienți de aproximare și un trece-sus g pentru coeficienți de detaliu, aplicate recursiv, dau analiza multi-rezoluție [4], punctul de plecare al Capitolului 3. Condiția care garantează reconstrucție perfectă e relația de filtre-oglină-conjugate, $g[k] = (-1)^k h[N-1-k]$ [5, 6], iar motivul pentru care decimarea cu doi după fiecare filtrare nu pierde informație vine direct din teorema eșantionării: o bandă de B Hz cere doar $2B$ eșantioane pe secundă [7], exact atât cât rămâne după ce fiecare filtru înjumătățește banda.

2.3 Convoluția

Definiția 1D, $(f * g)[n] = \sum_k f[k] g[n-k]$, stă la baza filtrelor de mai sus și a rețelelor convoluționale deopotrivă. Teorema convoluției, $f * g \leftrightarrow F \cdot G$ [3], leagă cele două: o convoluție în timp e o simplă înmulțire în frecvență, motivul pentru care FFT reduce complexitatea unei convoluții lungi de la $O(n^2)$ la $O(n \log n)$: transformi, înmulțești punct cu punct, transformi înapoi.

Extensia la imagini,

$$(I * K)[x, y] = \sum_{i, j} I[x-i, y-j] \cdot K[i, j],$$

rotește kernelul cu 180° (semnele minus). Bibliotecile de procesare de imagini, și demo-urile din tur, calculează de fapt *corelație*, cu $I[x+i, y+j]$, fără rotația kernelului: pentru kernele simetrice (blur, Gaussian, Laplacian) rezultatul e identic, iar diferența apare doar la kernele asimetrice, la Sobel de exemplu, unde rotația ar inversa semnul gradientului (Figura 2.4).

Convoluția în Imagini (2D)

Convoluția 2D extinde principiul 1D la imagini: kernelul alunecă peste imagine, iar fiecare pixel de ieșire este suma ponderată a vecinilor săi.

$$(I * K)[x, y] = \sum_{i, j} I[x-i, y-j] \cdot K[i, j]$$

CONVOLUȚIE 2D: SEMNELE MINUS ROTESC KERNELUL CU 180 DE GRADE

Bibliotecile de procesare de imagini - și demo-urile care urmează - calculează de fapt corelația, cu $I[x+i, y+j]$, adică fără rotația kernelului. Pentru kernelurile simetrice (blur, Gaussian, Laplacian) rezultatul este identic; diferența apare la kernelurile asimetrice, de exemplu Sobel, unde se inversează semnul gradientului.

Blur Gaussian

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Medierea vecinilor, cu pondere mai mare pe centru: netezire

Sharpen

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Amplifică diferența față de vecini: contur mai clar

Sobel X (asimetric)

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Gradient pe orizontală: detectează muchiile verticale

12 / 50

Figura 2.4: Convoluția 2D cu semnul corect și nota despre corelație versus convoluție, alături de trei kernele reprezentative: blur Gaussian, sharpen și Sobel X (asimetric).

2.4 Fourier în două dimensiuni

Înainte de a trece la kernele, turul face un pas lateral care merită explicat, pentru că e locul unde tot ce s-a spus despre semnale 1D se mută pe imagini. Transformata Fourier discretă se aplică la fel de bine unei matrice de pixeli, pe rânduri și apoi pe coloane, iar rezultatul e tot o matrice, de aceeași dimensiune, pe care o vedem ca imagine: spectrul de magnitudine (Figura 2.5). Backend-ul o calculează la cerere, prin `/api/fourier/image`, pe imaginea aleasă din listă.

Se citește altfel decât un spectru 1D. Frecvențele joase stau în centru, cele înalte spre margini, iar distanța față de centru înseamnă cât de repede se schimbă intensitatea, nu unde se întâmplă schimbarea. O zonă întinsă și uniformă, cerul sau un perete, produce energie concentrată în centru. Muchiile și textura fină împing energie spre margini, iar orientarea se păstrează: o textură cu dungi verticale lasă o urmă orizontală în spectru, perpendiculară pe dungi. Scara e logaritmică, altfel componenta continuă din centru ar fi singurul lucru vizibil.

Consecința practică leagă capitolul: un blur de telefon e un filtru trece-jos în două dimensiuni, iar o accentuare de contur e un trece-sus, exact aceleași două operații din Secțiunea 2.2, doar că acum tăietura se face pe un disc în jurul centrului, nu pe un interval de pe o axă. Kernelele din secțiunea următoare sunt fix aceste filtre, scrise în domeniul spațial.

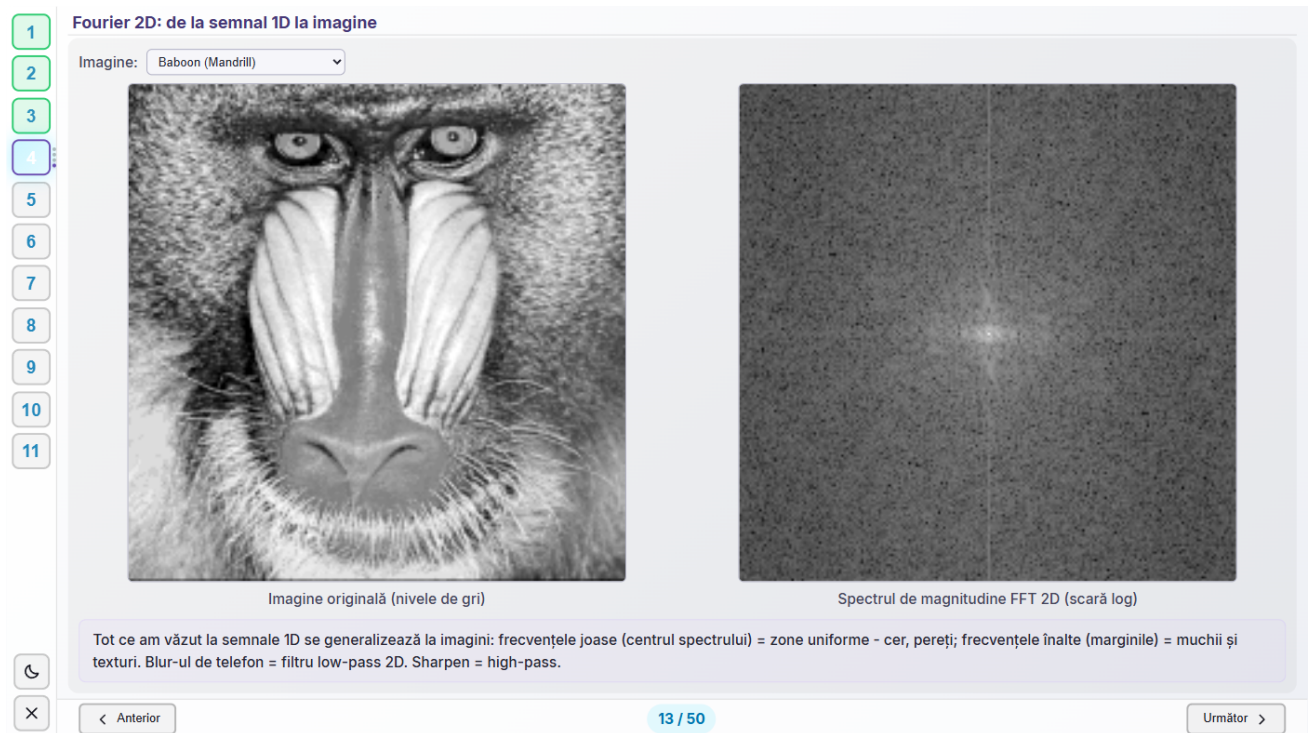


Figura 2.5: Aceeași imagine în cele două domenii: originalul în nivele de gri (stânga) și spectrul lui de magnitudine 2D pe scară logaritmică (dreapta), cu frecvențele joase în centru. Blana deasă a babuinului umple spectrul spre margini, semnătura unei texturi fine.

2.5 Kernele 2D, pe familii

Un demo educațional descompune aplicarea unui kernel pas cu pas, cu fereastra care alunecă, produsul element-cu-element și suma, înainte de a trece la kernele aplicate pe imagini reale (Figura 2.6): blur (cutie sau Gaussian), ascuțire (normală sau puternică), detectare de muchii (Laplacian, Laplacian diagonal, Sobel, Prewitt, contur) și emboss, fiecare cu rază (3, 4 sau 5) și puterea efectului reglabile independent.

Redimensionarea unui kernel la o rază mai mare trebuie să păstreze *ce face* kernelul, nu doar dimensiunea lui, iar o singură regulă nu ajunge pentru toate familiile. Identitatea rămâne o deltă discretă la orice dimensiune. Kernele de blur adevărat (toate valorile nenegative, suma 1) devin cutie sau Gaussian, după cum sursa era uniformă sau nu. Kernele de muchie (suma 0) păstrează tiparul 3×3 pe inelul exterior și rebalansează centrul ca suma să rămână 0 la orice dimensiune. Kernele direcționale (emboss) sunt interpolate bilinear și rescalate ca să păstreze suma originală, care le fixează luminozitatea rezultatului. Kernele de ascuțire simetrică, cu suma 1 dar și valori negative, deci nu blur, se reconstruiesc ca *unsharp masking* la noua rază,

$$K_{\text{sharpen}}(r) = (1 + \alpha) \delta_r - \alpha G_r,$$

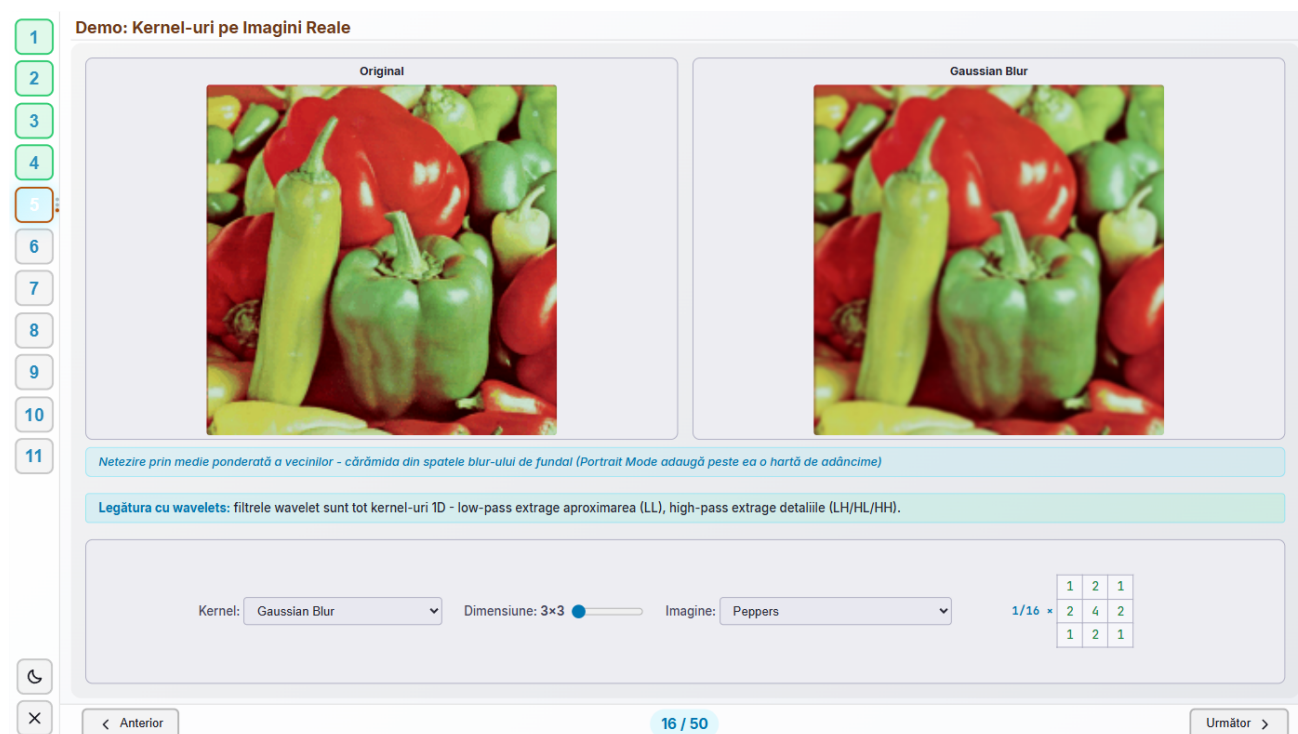


Figura 2.6: Blur Gaussian 3×3 aplicat pe imaginea Peppers, cu matricea de kernel afișată exact așa cum e trimisă către back-end.

unde δ_r e delta discretă, G_r Gaussiană normalizată la raza r , iar α vine din valoarea centrală a kernelului original minus 1, ca un sharpen la 3×3 și unul la 5×5 să păstreze aceeași putere a efectului.

Distincția pe familii a intrat în cod după ce o regulă mai simplă, „orice kernel cu suma 1 e un blur”, s-a dovedit greșită: și ascuțirea, și emboss-ul, și identitatea au suma 1, așa că un „Sharpen” cerut la 5×5 întorcea de fapt o gaussiană, cu un câștig măsurat de zero la frecvența Nyquist. Cu funcția rescrisă pe familii, pe o fotografie reală, varianța laplacianului imaginii, o măsură standard de claritate, urcă de la 2307 pe original la 40150, 33893 și respectiv 32114 pentru ascuțire la 3×3 , 5×5 și 7×7 .

Capitolul 3

Transformata wavelet și algoritmul lui Mallat

3.1 De ce wavelets

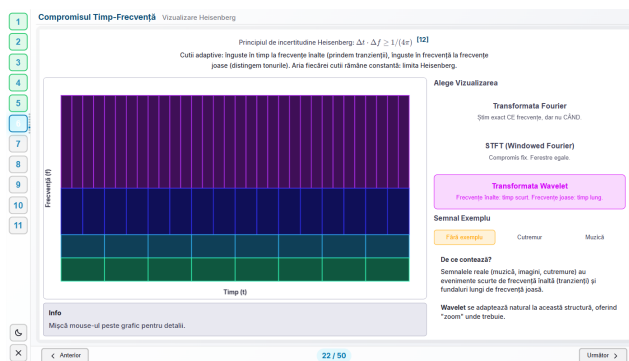
La un cutremur, unda P sosește la $t = 12s$ și unda S la $t = 38s$. Un spectru Fourier calculat pe tot semnalul nu poate spune când s-a schimbat regimul, ceea ce lipsește e localizarea. Wavelets rezolvă exact asta: în loc de o sinusoidă infinită, folosesc un „wavelet mamă” ψ , o undă scurtă, cu energie concentrată local, scalată și translatată,

$$\psi_{a,b}(t) = \frac{1}{\sqrt{|a|}} \psi\left(\frac{t-b}{a}\right),$$

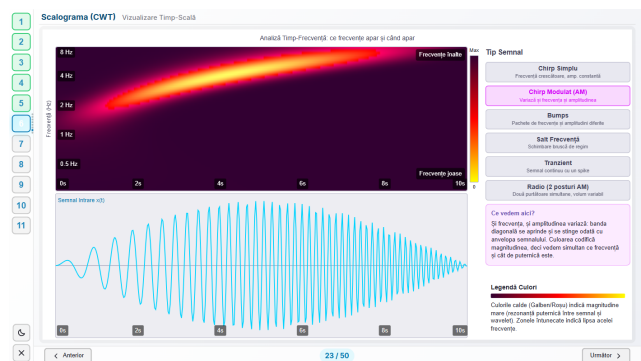
unde b e momentul de timp și a scala, invers proporțională cu frecvența, deci a mic prinde detalii de frecvență înaltă, a mare prinde structuri de frecvență joasă. Rezultatul e o analiză multi-rezoluție: semnale nestaționare (muzică, ECG, cutremure) capătă o descriere în care timpul și frecvența apar simultan.

Compromisul e formalizat de principiul de incertitudine al lui Heisenberg, $\Delta t \cdot \Delta f \geq 1/(4\pi)$ [12]: fiecare metodă împarte planul timp-frecvență în dreptunghiuri de arie constantă, dar forma lor diferă. Fourier pur dă rezoluție infinită în frecvență și nulă în timp. STFT, cu fereastră fixă, dă un compromis identic la orice frecvență. Wavelets adaptează fereastra: îngustă în timp la frecvențe înalte, îngustă în frecvență la frecvențe joase (Figura 3.1, stânga), forma care se potrivește cel mai bine cu semnale reale, unde tranzițiile scurte (un atac de tobă, un complex QRS) coexistă cu structuri lungi (un bas continuu, linia de bază a unui ECG).

Scalograma (Figura 3.1, dreapta) arată aceeași idee pe un semnal concret: transformata wavelet continuă a unui chirp modulat în amplitudine produce o bandă diagonală, frecvența crește cu timpul, a cărei intensitate urmărește chiar amplitudinea semnalului de intrare.



(a) Cutii de arie constantă: înguste în timp la frecvențe înalte, înguste în frecvență la frecvențe joase.



(b) Scalograma unui chirp modulat: banda diagonală urcă în frecvență și respiră odată cu amplitudinea.

Figura 3.1: Compromisul timp-frecvență, teoretic (stânga) și pe un semnal concret (dreapta).

3.2 Familii de wavelets

Nu orice undă scurtă e un wavelet valid. Condiția de admisibilitate cere ca $\int_0^\infty |\hat{\psi}(\omega)|^2/\omega d\omega$ să fie finită, ceea ce pentru un ψ integrabil forțează media lui nulă, $\int \psi(t) dt = 0$ [8], de-aici numele de familie: un wavelet oscilează în jurul lui zero, nu se așază pe o valoare. Funcția de scalare ϕ și wavelet-ul ψ se leagă recursiv prin coeficienții filtrului,

$$\phi(t) = \sqrt{2} \sum_k h_k \phi(2t - k), \quad \psi(t) = \sqrt{2} \sum_k g_k \phi(2t - k),$$

cu g obținut din h prin aceeași relație de filtre-oglină-conjugate din Secțiunea 2.2 [5, 6]. Familia Daubechies [9], Symlet, Coiflet, Morlet și altele diferă prin numărul de coeficienți și prin cât de simetrice sunt, toate schimbă doar ϕ și ψ , algoritmul de mai jos rămâne același.

Transformata continuă, $W(a, b) = \int f(t) \cdot \frac{1}{\sqrt{a}} \psi^*\left(\frac{t-b}{a}\right) dt$, evaluează (a, b) pe un continuu, deci informația se repetă. Transformata discretă eșantionează diadic, $a = 2^j$, $b = k \cdot 2^j$, și produce exact atâția coeficienți câți sunt necesari pentru o bază ortonormală, fără redundanță [12], baza pe care lucrează algoritmul lui Mallat, mai jos. Un demo din tur desenează live forma mamă pentru orice familie disponibilă în PyWavelets, citind direct rezultatul funcției `wavefun()` a bibliotecii. O sinusoidă simplă e inclusă explicit ca ne-exemplu, fiindcă n-are nicio localizare în timp, deci nu e wavelet.

The screenshot shows a web application interface for wavelet theory. It has a sidebar with numbered tabs (1-11) and a main content area with four panels, each with a title and text. The panels are:

- Media nulă (condiție necesară)**: $C_\psi = \int_0^\infty \frac{|\hat{\psi}(\omega)|^2}{\omega} d\omega < \infty \Rightarrow \int_{-\infty}^\infty \psi(t) dt = 0$. Condiția de admisibilitate cere integrala C_ψ finită; pentru un ψ integrabil ea forțează media nulă, deci wavelet-ul oscilează în jurul lui zero. [8]
- Ecuția de Scalare (Dilatație)**: $\phi(t) = \sqrt{2} \sum_k h_k \phi(2t - k)$. Funcția de scalare ϕ se exprimă recursiv prin coeficienții h .
- Ecuția Wavelet**: $\psi(t) = \sqrt{2} \sum_k g_k \phi(2t - k)$. Wavelet-ul ψ se derivă din ϕ folosind coeficienții g .
- Transformata Wavelet Continuă (CWT)**: $W(a, b) = \int_{-\infty}^\infty f(t) \cdot \frac{1}{\sqrt{a}} \psi^*\left(\frac{t-b}{a}\right) dt$. Evaluează (a, b) continuu: informația se repetă (redundant).
- Filtre în oglindă conjugate (CQF)**: $g_k = (-1)^k h_{N-1-k}$. High-pass-ul g se obține din low-pass-ul h prin inversare în timp și alternarea semnului (alternating flip). Numele istoric QMF descrie o altă pereche, care cu filtre FIR dă reconstrucție perfectă doar pentru Haar. [5] [6]
- De ce DWT nu e redundant**: $a = 2^j, b = k \cdot 2^j, j, k \in \mathbb{Z}$. CWT evaluează (a, b) continuu, deci informația se repetă. DWT eșantionează doar diadic (scală putere a lui 2, translație multiplu de scală): exact atâția coeficienți cât produc o bază ortonormală, fără redundanță. [12]

Figura 3.2: Cele patru rezultate din spatele familiilor wavelet: media nulă, ecuațiile de scalare/wavelet, perechea CQF și non-redundanța DWT, fiecare cu citarea lui.

3.3 Wavelet-ul complex și faza

Familiiile de până acum sunt reale: un coeficient mare poate să însemne rezonanță puternică, dar semnul lui depinde de unde anume a căzut wavelet-ul peste oscilație. Mută semnalul cu o fracțiune de perioadă și coeficientul se schimbă, deși conținutul nu s-a schimbat deloc. Wavelet-ul Morlet complex rezolvă exact asta, pentru că e o gaussiană înmulțită cu o exponențială complexă,

$$\psi(t) = e^{-t^2/2} \cdot e^{i\omega t} = e^{-t^2/2} (\cos(\omega t) + i \sin(\omega t)),$$

adică două wavelet-uri reale defazate cu un sfert de perioadă, folosite împreună. Coeficientul devine un număr complex, din care se citesc separat magnitudinea, care spune *cât* de puternică e rezonanța și nu se mai schimbă la deplasarea semnalului, și faza, care spune *unde* a căzut oscilația în interiorul ferestrei.

Ecranul din tur (Figura 3.3) arată wavelet-ul ca o spirală în trei dimensiuni: timpul pe o axă, partea reală pe a doua, partea imaginară pe a treia, cu culoarea codificând faza de la $-\pi$ la π . Proiecția pe fiecare perete dă cele două componente reale, iar spirala e motivul pentru care scalograma din secțiunea anterioară arată benzi netede și nu o alternanță de dungi luminoase și întunecate: acolo se desenează magnitudinea coeficientului complex, care nu depinde de fază.

Aceeași proprietate e motivul pentru care wavelet-urile complexe se folosesc la detectarea trăsăturilor invariante la translație și la analiza mișcării, unde contează ce s-a întâmplat, nu la ce microsecundă a căzut fereastra.



Figura 3.3: Wavelet-ul Morlet complex ca spirală: partea reală proiectată pe un perete, partea imaginară pe celălalt, iar culoarea curbei centrale codifică faza. Frecvența și unghiul de rotație sunt reglabile din dreapta.

3.4 Algoritmul lui Mallat

Formal, un semnal se descompune prin proiecție pe funcția de scalare și pe wavelet la nivelul j ,

$$c_{j,k} = \int x(t) \phi_{j,k}(t) dt, \quad d_{j,k} = \int x(t) \psi_{j,k}(t) dt.$$

Contribuția lui Mallat e observația care face aceste integrale inutile de calculat direct: coeficienții unui nivel se obțin din nivelul anterior doar prin filtrare și decimare cu doi,

$$c_{j+1,k} = \sum_n h_0[n - 2k] c_{j,n}, \quad d_{j+1,k} = \sum_n h_1[n - 2k] c_{j,n},$$

unde h_0 e filtrul de aproximare h și h_1 filtrul de detaliu g din ecuațiile de mai sus, scrise acum cu indici fiindcă apar împreună, fără nicio integrală calculată efectiv [13, 12]. E o recursie exactă: fiecare nivel iese din cel anterior printr-o singură trecere de filtrare, deci descompunerea pe L niveluri costă L perechi de filtrări, nu L integrale separate. În cod, aceeași recursie e `pywt.wavedec2(imagine, wavelet, level=niveluri)`, care întoarce aproximarea cea mai grosieră plus câte un triplet de subbenzi de detaliu pe nivel. Reconstrucția, `pywt.waverec2`, trece coeficienții prin filtrele de sinteză și îi însumează, operația inversă a celei de mai sus.

Pentru wavelet-ul Haar, singurul cu filtre de doi coeficienți, suma se scrie pe un singur eșantion și devine o pereche de medie și diferență,

$$x[2k] = \frac{cA[k] + cD[k]}{\sqrt{2}}, \quad x[2k+1] = \frac{cA[k] - cD[k]}{\sqrt{2}},$$

forma folosită de demo-ul de reconstrucție din tur. Pentru orice filtru mai lung, un Daubechies sau biortogonalul 9/7 din capitolul de compresie, un eșantion reconstruit primește contribuții de la mai multe perechi (cA, cD) vecine, iar formula de mai sus nu se mai aplică punct cu punct.

La imagini, aceeași pereche de filtre se aplică o dată pe rânduri și o dată pe coloane, dând patru subbenzi, LL (aproximare), LH/HL (detalii orizontale/verticale) și HH (detalii diagonale). Analogia cu hărțile ajută: la zoom mic vezi doar LL , structura globală. La zoom mare apar $LH/HL/HH$, detaliile fine. Pasul se aplică recursiv doar pe LL . High-pass pe coloane dă detalii *orizontale* pentru că răspunde la variația pe verticală, exact ce înseamnă o muchie orizontală.

Demo-ul educațional 1D (Figura 3.4) face vizibil un nivel al algoritmului, pas cu pas, pe un semnal de 16 eșantioane, treaptă, rampă, sinusoidă, puls sau o „melodie” de note discrete. Fereastra filtrului, produsul și suma sunt animate explicit, apoi decimarea cu doi. Interpretarea diferă cu forma semnalului, o treaptă produce un vârf de detaliu exact la muchie, o melodie păstrează notele în aproximare și marchează atacul fiecărei note în detaliu. Un glosar de notație, repetat pe toate ecranele acestei secțiuni, ține evidența faptului că aceeași împărțire circulă sub nume diferite în literatură și în cod: $c = cA = L = A$ pentru aproximare, $d = cD = H = D$ pentru detaliu.

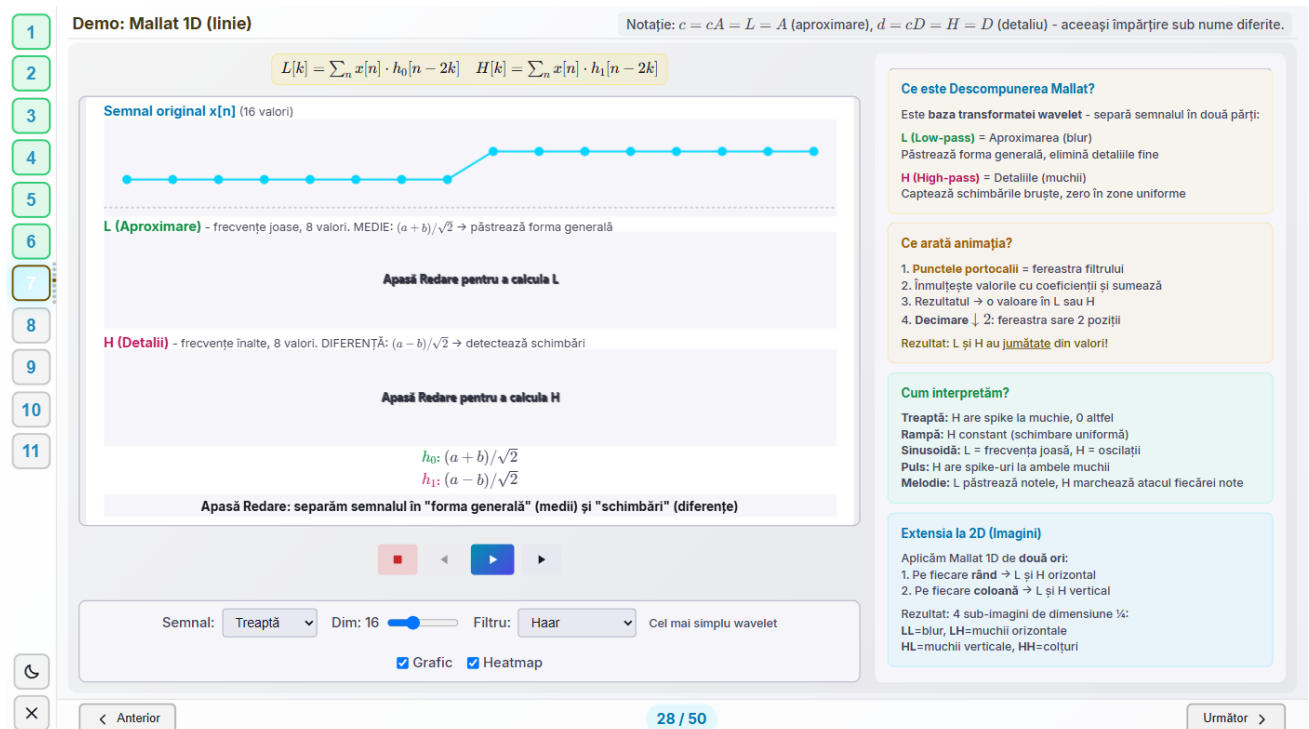
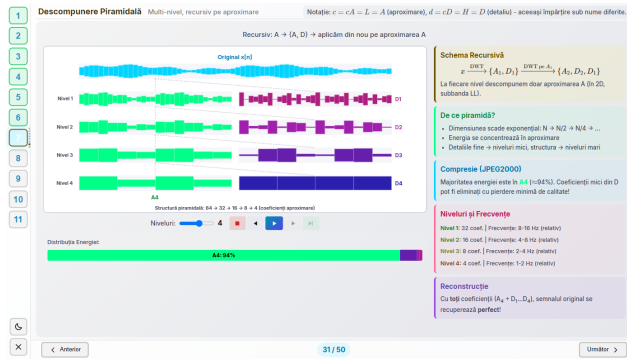


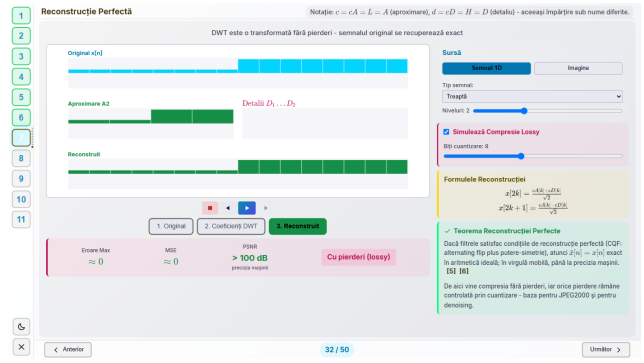
Figura 3.4: Un nivel al algoritmului lui Mallat pas cu pas: fereastra filtrului, produsul, suma și decimarea cu doi, pe un semnal treaptă de 16 eșantioane.

Aplicat recursiv doar pe aproximare, pe patru niveluri (Figura 3.5, stânga), pasul arată unde se duce energia unui semnal real: peste 94% rămâne concentrată în aproximarea cea mai grosieră, A4. E numărul concret din spatele afirmației că „majoritatea energiei unei imagini încapă într-un număr mic de coeficienți”, premisa pe care se sprijină toată povestea compresiei din Capitolul 4. Un ultim ecran (Figura 3.5, dreapta) arată ce înseamnă „reconstrucție perfectă” în aritmetică reală: dacă filtrele satisfac condiția CQF, eroarea rămâne la precizia mașinii de calcul, iar PSNR-ul raportat reflectă exact acea precizie finită, de aici pornește atât compresia *lossless*, cât și orice pierdere ulterioară, controlată prin cuantizare.

Pentru compunerea vizuală a subbenzilor într-un singur mozaic, cum apare pe mai multe ecrane



(a) Patru niveluri recursive: peste 94% din energie rămâne în A4.



(b) Reconstrucție perfectă: eroarea rămâne la precizia mașinii, nu la zero exact.

Figura 3.5: Multi-nivel (stânga) și reconstrucția care închide bucla (dreapta).

din tur, fiecare pereche de subbenzi e decupată la dimensiunea minimă comună înainte de alăturare. PyWavelets poate produce subbenzi care diferă cu un pixel pe dimensiuni impare de intrare.

Capitolul 4

Compresia imaginilor: JPEG versus JPEG2000

Comprimarea unei imagini se reduce, în ambele standarde, la aceeași idee de la finalul capitolului trecut: transformă, păstrează coeficienții mari, taie sau cuantizează grosier restul. JPEG transformă cu DCT pe blocuri mici. JPEG2000 transformă cu DWT, pe toată imaginea, folosind exact algoritmul lui Mallat. Acest capitol arată pipeline-ul JPEG pas cu pas, apoi pune cele două transformate față în față, la bitrate egal, cu cifre măsurate.

4.1 Pipeline-ul JPEG

Baseline JPEG parcurge șase etape: conversie RGB în YCbCr (separă luminanța de cromatică), subeșantionare de cromatică (ochiul distinge culoarea mai slab decât luminozitatea), DCT pe blocuri de 8×8 , cuantizare cu tabelul Annex K scalat după regula IJG, apoi zigzag și codare pe lungimi de serie. Demo-ul din tur (Figura 4.1) rulează toate șase pe o fotografie reală, nu pe un gradient sintetic, un gradient neted n-are ce pierde la subeșantionare, deci ar ascunde exact etapa pe care vrea s-o arate.



Figura 4.1: Prima etapă a pipeline-ului JPEG pe o fotografie reală: separarea Y/Cb/Cr, cu formulele de conversie folosite efectiv de back-end.

Cele două standarde lasă în urmă tipuri diferite de artefact la compresie mare. JPEG, cu blocuri

independente, lasă blocare: grila de 8×8 devine vizibilă. JPEG2000, cu o transformată globală, lasă *ringing* [10]: oscilații fine în jurul muchiilor, nu o grilă. JPEG e progresiv doar dacă fișierul e codat explicit în acel mod din T.81, baseline-ul, cel mai comun pe web, se decodează secvențial [17]. JPEG2000 capătă scalabilitate progresivă direct din construcția multi-rezoluție: aceeași descompunere care dă comprimarea dă și un flux din care poți citi mai întâi o versiune la rezoluție mică, apoi detaliile [10].

4.2 O comparație la bitrate egal

O primă versiune a acestei comparații compara cele două codecuri la un parametru de „calitate” egal, calibrat diferit pe fiecare parte, partea DCT folosea tabelul real Annex K, partea wavelet un prag liniar ad-hoc, fără nicio legătură cu costul în biți, și scotea un rezultat înșelător, cu JPEG câștigând la orice calitate testată. Comparația de mai jos egalează bitrate-ul, nu eticheta de calitate, și măsoară amândouă codecurile cu același estimator de cost.

Costul în biți. Fiecare codec e măsurat cu entropia de ordin 0 a simbolurilor lui cuantizate, înmulțită cu numărul lor,

$$R = N \cdot H(\mathbf{p}), \quad H(\mathbf{p}) = - \sum_i p_i \log_2 p_i,$$

unde p_i e frecvența empirică a simbolului i în histograma coeficienților cuantizați. Un JPEG sau un JPEG2000 adevărat iese mai mic decât atât, pentru că folosește o codare entropică reală. Huffman, aritmetică sau, la JPEG2000, EBCOT cu context, dar cum ambele părți sunt măsurate cu același estimator, raportul dintre cele două costuri rămâne comparabil, chiar dacă niciuna dintre cifrele absolute nu e dimensiunea unui fișier real.

Partea DCT. Imaginea se taie în blocuri de 8×8 pixeli, iar fiecare bloc trece prin transformata de cosinus discretă ortonormală,

$$X_{u,v} = \frac{1}{4} C_u C_v \sum_{i=0}^7 \sum_{j=0}^7 x_{i,j} \cos \left[\frac{(2i+1)u\pi}{16} \right] \cos \left[\frac{(2j+1)v\pi}{16} \right], \quad C_0 = \frac{1}{\sqrt{2}}, \quad C_{k>0} = 1,$$

care scrie blocul ca sumă de 64 tipare de cosinus, de la cel constant la cel mai alternant. Urmează cuantizarea cu tabelul de luminanță Annex K scalat după regula IJG,

$$\text{scală} = \begin{cases} 5000/\text{calitate} & \text{calitate} < 50 \\ 200 - 2 \cdot \text{calitate} & \text{calitate} \geq 50 \end{cases}, \quad Q = \text{clip} \left(\left\lfloor \frac{Q_{50} \cdot \text{scală} + 50}{100} \right\rfloor, 1, 255 \right).$$

Partea wavelet primește bugetul de biți al părții DCT. În loc de propriul parametru de calitate, partea wavelet caută prin biseție geometrică pasul de cuantizare Δ cel mai fin care încă încapă în bugetul de biți al părții DCT (treizeci de iterații, interval de căutare [0.02, 8192], oprire la o precizie relativă de 0.2%). Cuantizarea e cu *zonă moartă*, cu un pas propriu pe fiecare subbandă b ,

$$q_b = \text{sign}(c_b) \left\lfloor \frac{|c_b|}{\Delta/g_b} \right\rfloor, \quad \hat{c}_b = \text{sign}(q_b) \left(|q_b| + \frac{1}{2} \right) \frac{\Delta}{g_b},$$

cu reconstrucția la *mijlocul* intervalului de cuantizare, convenția standard JPEG2000. Factorul g_b e câștigul de sinteză al subbenzii: transformata folosită implicit, **bior4.4**, e chiar filtrul 9/7 al JPEG2000, construit istoric prin schema de *lifting* [11], iar fiind biortogonală, nu ortonormală, o eroare de un coeficient nu produce aceeași eroare de reconstrucție în toate subbenzile. Câștigul g_b se măsoară direct, nu prin formulă: un impuls unitar plasat în centrul subbenzii, trecut prin transformata inversă singur, iar norma L_2 a rezultatului e g_b , exact tehnica JPEG2000 de pași de cuantizare diferiți pe subbandă [10], aplicată empiric.

Metrici. PSNR e definiția uzuală, $\text{PSNR} = 10 \log_{10}(255^2/\text{MSE})$. SSIM e implementarea proprie a formulei lui Wang și colaboratorii,

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \quad C_1 = (0.01L)^2, \quad C_2 = (0.03L)^2, \quad L = 255,$$

pe ferestre 7×7 [18], verificată numeric față de `skimage.metrics.structural_similarity`: 0.861644 (implementare proprie) față de 0.861132 (referință) pe imaginea `kodim01` la calitate 30, diferența venind din tratarea marginilor ferestrei.

Măsurători. Tabelul 4.1 arată diferența wavelet minus DCT, la bitrate egal, pentru `bior4.4` pe patru niveluri, pe trei imagini standard de test.

Tabela 4.1: Diferența wavelet minus DCT la bitrate egal (`bior4.4`, 4 niveluri).

Imagine	Calitate	bpp	ΔPSNR	ΔSSIM
<code>kodim01</code> (768×512)	10	0.512	+0.23 dB	−0.026
<code>kodim01</code>	20	0.865	+0.74 dB	−0.007
<code>kodim01</code>	30	1.132	+1.12 dB	+0.003
<code>kodim01</code>	50	1.522	+1.71 dB	+0.010
<code>kodim01</code>	75	2.159	+2.55 dB	+0.012
<code>peppers_512</code>	10	0.537	−0.33 dB	−0.017
<code>peppers_512</code>	30	1.016	+0.42 dB	−0.011
<code>peppers_512</code>	50	1.319	+0.90 dB	−0.005
<code>baboon_512</code>	10	0.484	+0.12 dB	−0.036
<code>baboon_512</code>	30	1.135	+0.96 dB	+0.006
<code>baboon_512</code>	50	1.567	+1.59 dB	+0.015

Notă: în ciuda numelui de fișier, `peppers_512`, `baboon_512`, `house_512` și `lake_512` au de fapt 200×200 pixeli.

Demo-ul din tur (Figura 4.2) rulează exact această comparație, live, pe imaginea și calitatea alese: la aproximativ 1.01 biți pe pixel, wavelet câștigă pe PSNR (+0.93 dB) și pierde la limită pe SSIM (−0.001), cu ambele metrice afișate pe ecran.

Tabelul spune ceva simplu: wavelet câștigă pe PSNR aproape peste tot, cu marja crescând odată cu bitrate-ul. SSIM îl favorizează ușor pe JPEG la bitrate foarte mic, și e un rezultat real, nu o eroare de măsurare, tabelul Annex K e ponderat perceptual, construit din studii asupra sensibilității la contrast a ochiului uman, în timp ce cuantizatorul wavelet folosit aici e optim pe eroare medie pătratică, nu pe percepție. Restul avantajului pe care JPEG2000 îl are în literatură la bitrate mic vine din codarea entropică cu context, EBCOT [10], pe care estimatorul de entropie de ordin 0 folosit aici nu îl modelează: comparația izolează transformata și cuantizarea, nu codecul complet.

DCT (JPEG) vs Wavelet (JPEG2000) Compare la același bitrate

la același bpp (~0.93 est.)

DCT/JPEG: Blocuri 8×8 - artefacte de blocare **Wavelet/JPEG2000:** Global multi-rezoluție - fără blocuri

PSNR = $10 \log_{10} \frac{MAX^2}{MSE}$ [10]
 SSIM compară structura locală, nu eroarea punct cu punct [18]

DCT: Artefacte de bloc la margini Wavelet: gradienti fini, fără blocuri Calitate 20-30 - diferențe vizibile!

Original

DCT (JPEG)

SSIM 0.907 | 0.93 bpp | MSE 50.7

Wavelet (bior4.4)

SSIM 0.896 | 0.92 bpp | MSE 47.5

PSNR: Wavelet +0.28 dB SSIM: DCT +0.011 la ~0.93 bpp (est.)
 PSNR și SSIM nu sunt de acord: Anexa K din JPEG e ponderată perceptual, cuantizarea wavelet de aici optimizează MSE.

> PSNR vs Calitate - la 10%: DCT +0.33 dB

Zoom pe muchie Imagine: Kodak 01 - Stone Building Calitate: 25% Wavelet: Biorthogonal 9/7 Nivele: 4 Run

< Anterior 47 / 50 Următor >

Figura 4.2: Comparație la bitrate egal (≈ 1.01 bpp): wavelet câștigă pe PSNR și pierde la limită pe SSIM, cu ambele metrice afișate pe ecran.

Capitolul 5

Denoising prin prag

Zgomotul produce coeficienți wavelet mici. Semnalul util produce coeficienți mari. Un prag care taie sau atenuează coeficienții mici, apoi reconstruiește, elimină zgomotul fără să tocească muchiile, spre deosebire de un filtru trece-jos clasic, care ar netezi și detaliile utile odată cu zgomotul.

Estimarea zgomotului folosește abaterea mediană absolută a subbenzii celei mai fine de detaliu,

$$\hat{\sigma} = \frac{\text{median}(|d_{\text{fin}}|)}{0.6745},$$

o alegere robustă: spre deosebire de o deviație standard calculată direct, nu e trasă în sus de coeficienții mari care poartă semnal. Pragul universal al lui Donoho și Johnstone, $\lambda_{\text{universal}} = \hat{\sigma}\sqrt{2 \ln N}$ [16], iar cele două reguli de trunchiere sunt

$$\eta_{\text{soft}}(x, \lambda) = \text{sign}(x) \max(|x| - \lambda, 0), \quad \eta_{\text{hard}}(x, \lambda) = x \cdot \mathbb{1}(|x| > \lambda).$$

Soft micșorează continuu coeficienții spre zero, fără discontinuități. Hard elimină complet ce e sub prag și păstrează exact restul, cu riscul unor mici artefacte la graniță (Figura 5.1).

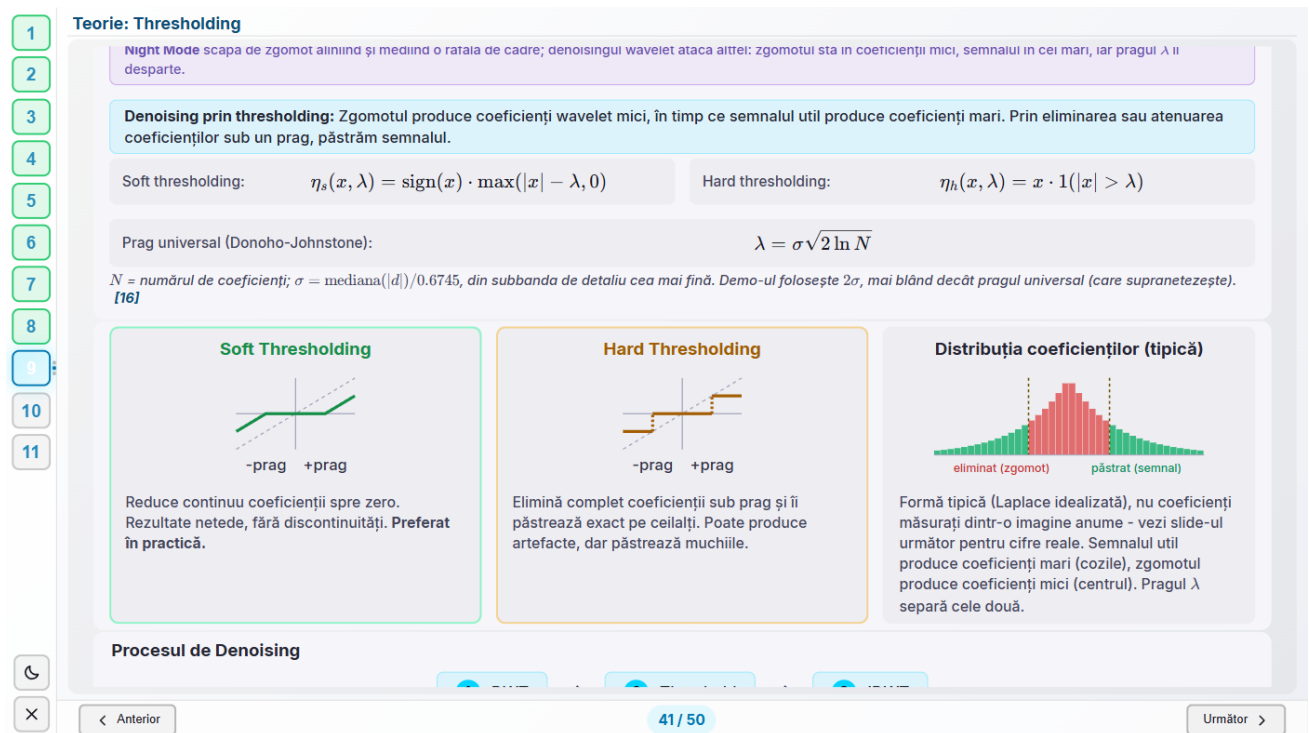


Figura 5.1: Thresholding moale versus dur și distribuția tipică a coeficienților, cu pragul universal Donoho-Johnstone alături de pragul $2\hat{\sigma}$ folosit efectiv de demo.

Pragul folosit efectiv pe imagini e $2\hat{\sigma}$, mai mic decât pragul universal din formulă. Abaterea e deliberată și documentată direct în cod. La $\hat{\sigma} = 15$ și $N = 512^2$ (o imagine 512×512), pragul universal

iese în jur de 68, atât de agresiv încât câștigul de raport semnal-zgomot scade spre 0 dB: teoria din spatele lui presupune un semnal neted plus zgomot alb, o ipoteză prea optimistă pentru texturile unei fotografii. Pragul 2 $\hat{\sigma}$ păstrează detaliul și obține în jur de 3 dB câștig. Demo-ul de ECG (Capitolul 6) folosește în schimb chiar pragul universal, pe un semnal 1D sintetic, unde ipoteza semnal neted plus zgomot alb se potrivește mult mai bine decât pe o imagine.

Pe o imagine reală (Figura 5.2), același proces rulează cu cifrele la vedere: dincolo de imaginea curățată apare și ce anume s-a eliminat, amplificat de zece ori, zgomot granular pur, fără nicio structură a imaginii originale, semn că a rămas doar semnalul.

1
2
3
4
5
6
7
8
9
10
11

Demo: Denoising Practic

Aceeași problemă ca la Night Mode pe telefon: o fotografie plină de granulație. Aici o rezolvăm pe calea wavelet: zgomotul trăiește în coeficienții mici (variații fine, aleatorii), algoritmul îi taie sub un prag și reconstruiește imaginea curată.

WAVELET
Daubechies 4

NIVELURI: 3

PRAG
Soft (moale)

ZGOMOT Σ : 15

IMAGINE
Peppers

Aplică

Original

Cu zgomot ($\sigma=15$)

Curățat de zgomot (soft)

Ce s-a eliminat (x10)

SNR Ințial 18.9 dB

SNR Final 22.1 dB

Prag automat (2σ) 29.83

Îmbunătățire +3.23 dB

$\lambda = 2\hat{\sigma} = 29.8, \quad \hat{\sigma} = \frac{\text{median}(|HH_1|)}{0.6745} = 14.9$

În cuvinte: semnalul util este acum de 2.1 ori mai puternic față de zgomot decât înainte (18.9 dB $\rightarrow$ 22.1 dB).
Imaginea "Ce s-a eliminat" arată exact ce a tăiat pragul: zgomot granular, fără structuri ale imaginii - dovada că am eliminat zgomotul, nu detaliile.

↻

×

< Anterior

42 / 50

Următor >

Figura 5.2: Denoising pe o imagine reală: reziduul amplificat $\times 10$ e zgomot granular pur, fără nicio structură a imaginii, semn că a rămas doar zgomotul.

Capitolul 6

Aplicație: electrocardiograma

O bătaie de inimă, văzută pe un ECG, e o secvență de unde cu nume proprii: P marchează depolarizarea atriilor, complexul QRS depolarizarea rapidă a ventriculilor, vârful ascuțit pe care oricine îl recunoaște dintr-un ECG, iar T repolarizarea ventriculilor. Diagnosticul clinic se citește din formă, din durată și din regularitatea intervalelor dintre bătăi, toate trei cer, mai întâi, să găsești fiecare complex QRS într-un semnal care aproape niciodată nu e curat.

6.1 De ce wavelets, aici

Un complex QRS e o tranziție bruscă de amplitudine, o *singularitate* locală, în limbajul procesării de semnal, suprapusă peste zgomot muscular, interferență electrică și o linie de bază care alunecă lent. Fourier nu ajută direct: spune ce frecvențe conține semnalul, nu unde anume apare tranziția. O transformată wavelet, la o scală apropiată de durata tipică a unui QRS ($\sim 0.08-0.12s$), face exact ce trebuie: coeficienții mari apar acolo unde semnalul se schimbă brusc, restul rămâne mic. E aceeași proprietate de localizare din Capitolul 3, aplicată la un semnal 1D real.

Martinez, Almeida, Olmos, Rocha și Laguna [15] formalizează ideea într-un algoritm în trei etape: mai întâi detectează complexe QRS, apoi delimitează fiecare complex, vârfurile undelor individuale, începutul și sfârșitul lui, și în final localizează undele P și T. Mecanismul de bază e detecția de singularități prin maxime de modul ale transformatei wavelet pe mai multe scări: un wavelet construit ca derivata unei funcții de netezire produce, lângă o tranziție bruscă, o pereche de extreme de semn opus, iar tranziția însăși cade aproape de trecerea prin zero dintre ele, teoria generală de detecție a singularităților din [13, 12], aplicată aici la ECG. Pe bazele de date standard adnotate manual (MIT-BIH Arrhythmia, QT, European ST-T, CSE), detectorul de QRS obține o sensibilitate de 99.66% și o predictivitate pozitivă de 99.56%, din o mie de bătăi reale, sub patru sunt ratate sau inventate.

Miza practică e directă: un ceas care anunță fibrilație atrială citește un semnal de fotopletismografie (PPG) în locul unui ECG de spital. E lumină reflectată de piele, modulată de pulsul sangvin, din care extrage neregularitatea intervalelor dintre bătăi, uneori completat cu o singură derivație de ECG pe electrozi din carcasă [14]. Fie că sursa e un ECG de spital pe douăsprezece derivații sau un semnal de încheietură zgomotos, primul pas practic e același: găsește bătăile.

6.2 Demo

Demo-ul din tur (Figura 6.1) construiește un ECG sintetic ca sumă de gaussiene pentru undele P, Q, R, S, T ale fiecărei bătăi, o aproximare standard pentru semnale de test, îi adaugă zgomot muscular reglabil, îl curăță cu o transformată wavelet 1D (Capitolul 5, cu pragul universal Donoho-Johnstone, potrivit aici pentru că semnalul e aproape de ipoteza „neted plus zgomot alb”) și detectează vârfurile R ca maxim pe fereastra fiecărei bătăi. Frecvența cardiacă iese din mediana intervalelor R-R. În captura din figură, cu zgomotul la $\sigma = 0.25$, toate cele șase bătăi sunt găsite corect, iar raportul semnal-zgomot urcă de la -2.9 la 2.1 dB.

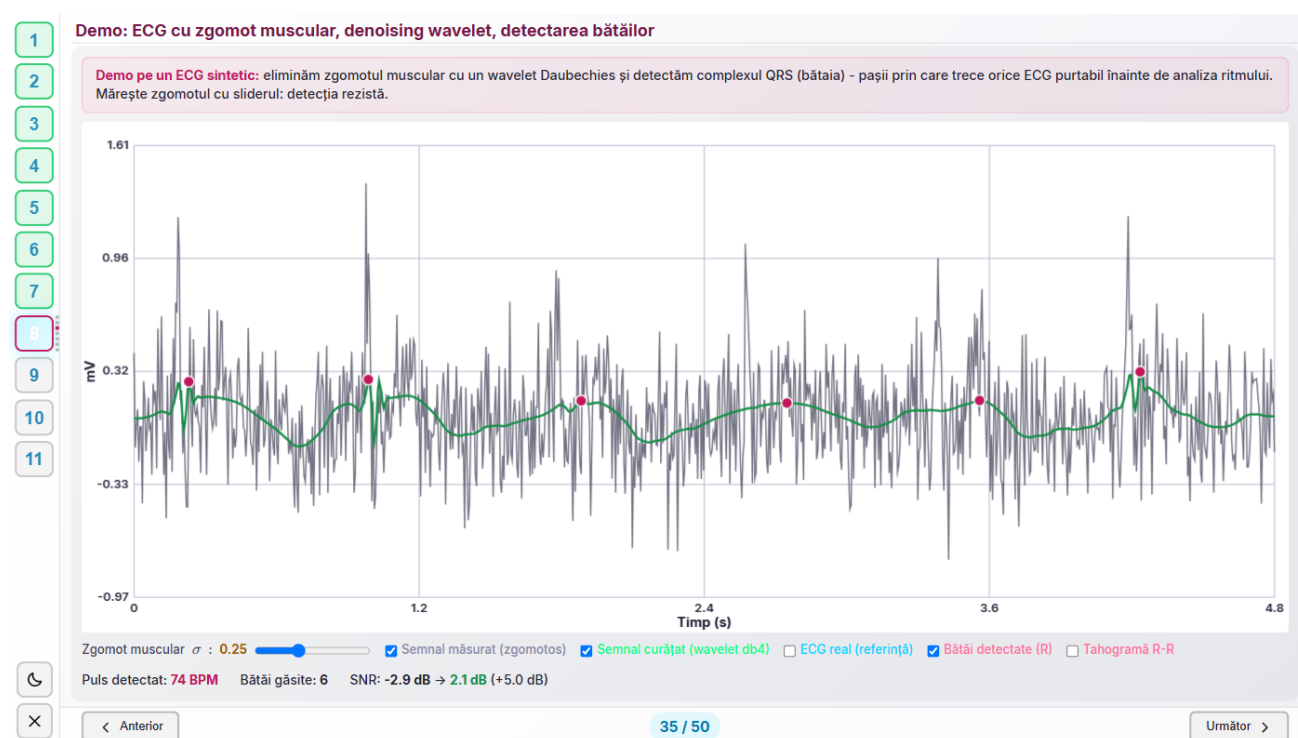


Figura 6.1: ECG sintetic cu zgomot muscular: denoising wavelet și detecția celor șase băți, SNR de la -2.9 la 2.1 dB.

Capitolul 7

Aplicație: EEG

Activitatea electrică a creierului, culeasă cu electrozi pe scalp, se citește pe benzi de frecvență cu semnificație clinică proprie: delta (0.5-4 Hz, somn profund), theta (4-8 Hz, relaxare adâncă), alfa (8-13 Hz, ochii închiși, relaxare trează), beta (13-30 Hz, concentrare activă) și gamma (30-80 Hz, procesare cognitivă complexă). Polisomnografia (studiul somnului), diagnosticul de epilepsie și interfețele creier-calculator se bazează, în forme diferite, pe separarea acestor benzi, un ceas care „estimează” somnul din mișcare și puls nu are acces la niciuna dintre ele. EEG-ul adevărat rămâne un instrument de laborator sau de clinică.

Demo-ul din tur (Figura 7.1) extrage o singură bandă dintr-un semnal EEG sintetic cu un filtru trece-bandă obișnuit (Secțiunea 2.2), nu cu transformata wavelet discretă folosită în restul documentului. Motivul e scris direct pe ecran: DWT dublează frecvența la fiecare nivel, o octavă, dar granițele benzilor clinice nu sunt spațiate octavă-cu-octavă, delta-theta e aproape un factor de doi, delta-alfa e deja un factor de 3.25, așa că niciun set fix de niveluri DWT nu s-ar alinia cu toate granițele deodată. Un filtru trece-bandă clasic, cu tăieturi alese liber, nu are această constrângere.



Figura 7.1: Banda alfa (8-13 Hz) extrasă cu un filtru trece-bandă dintr-un EEG sintetic. Nota din josul figurii explică de ce nu DWT.

Capitolul 8

Aplicație: recunoașterea muzicii

Recunoașterea unei piese din câteva secunde de audio, înregistrate cu un microfon de telefon, peste zgomot de fundal și voci, pare să ceară o comparație directă cu milioane de piese dintr-o bază de date. Algoritmul din spatele Shazam [2] evită exact asta, comparând amprente scurte și discrete în loc de audio brut.

8.1 Cum funcționează amprenta

Primul pas e o spectrogramă a piesei, exact tipul de reprezentare timp-frecvență din Capitolul 3. Din ea se păstrează doar vârfurile locale de energie, cele mai puternice puncte din fiecare regiune de timp-frecvență, alese cu un criteriu de densitate care le distribuie uniform pe toată durata piesei, cele mai tari câteva din fiecare fereastră, nu doar cele mai tari câteva sute din tot semnalul. Rezultatul e o „constelație” de puncte (t, f) , suficient de rară ca să încapă compact, suficient de densă ca să supraviețuiască zgomotului.

Un singur punct din constelație nu spune mare lucru, miile de piese din bază conțin, fiecare, vârfuri la frecvențe apropiate. Amprenta reală vine din *perechi*: fiecare punct devine un „punct-ancoră”, legat de câteva puncte dintr-o „zonă țintă” apropiată în timp și frecvență. Fiecare pereche (ancoră, țintă) generează un hash format din trei numere, frecvența ancorei, frecvența țintei și diferența de timp dintre ele, $(f_1, f_2, \Delta t)$, stocat împreună cu timpul absolut la care apare în piesă. O piesă de câteva minute produce astfel zeci de mii de hash-uri, nu un singur „cod” global.

Căutarea unui fragment înregistrat repetă exact procesul, spectrogramă, constelație, hash-uri, și caută fiecare hash în baza de date. Zgomotul de fundal adaugă vârfuri false și poate ascunde unele reale, dar tot lasă destule perechi curate ca zeci de hash-uri să se potrivească exact cu o singură piesă din bază, iar dacă hash-urile potrivite cad la o diferență de timp *constantă* față de timpul lor din piesa originală (fragmentul înregistrat e, de exemplu, exact piesa originală începând din secunda 47), potrivirea e confirmată statistic, nu doar prin numărul de hash-uri comune. Pentru că amprenta e locală, fiecare hash descrie doar o mică vecinătate timp-frecvență, metoda tolerează inclusiv piese suprapuse: hash-urile unei piese ascultate peste alta rămân, în bună parte, neschimbate, deci ambele pot fi recunoscute din același fragment.

8.2 Demo

Demo-ul din tur (Figura 8.1) arată, mai simplu, punctul de plecare al ideii: trei acorduri redade pe rând interferează în domeniul timp, iar semnalul mixt arată vizual de ce identificarea nu se poate face direct pe formă de undă, notele individuale nu mai sunt separabile cu ochiul. Textul de pe ecran leagă figura de mecanismul real: vârfurile dintr-o spectrogramă devin perechi $(f_1, f_2, \Delta t)$ căutate într-o bază de date, exact procesul descris mai sus.



Figura 8.1: Trei acorduri interferând în domeniul timp. Textul explică amprentele $(f_1, f_2, \Delta t)$ pe care le caută algoritmul real.

Capitolul 9

Sinteză: un singur mecanism, multe aplicații

Dincolo de ECG (Capitolul 6), EEG (Capitolul 7) și recunoașterea muzicii (Capitolul 8), aceeași transformată apare în audio (compresie, anularea zgomotului în căști), în securitate cibernetică (steganografie și *watermarking*, mesaje ascunse în coeficienții HH, invizibili ochiului), în finanțe (analiza volatilității pe scări de timp), în seismologie (separarea undelor P și S) și în astronomie (analiza semnalelor cosmice slabe, îngropate în zgomot).

Privit dintr-o parte, mecanismul e o alegere de dicționar. O bază wavelet e un dicționar fix, construit dinainte din condiții matematice, în care semnalele netede pe porțiuni au reprezentări rare, adică puțini coeficienți mari și restul aproape de zero. Toate capitolele de mai sus exploatează exact această raritate, fie tăind coeficienții mici ca zgomot, fie cheltuind biți doar pe cei mari. Întrebarea firească de aici încolo e dacă dicționarul trebuie ales dinainte sau poate fi învățat din datele înseși, ceea ce duce la reprezentări rare cu dicționare antrenate, unde baza se învață din exemplele pe care urmează să le comprime, în locul unei construcții pornite din condiții teoretice.

Ce leagă toate aplicațiile din acest document e un singur gest, repetat: descompune semnalul pe scări (aceeași bancă de filtre din Secțiunea 3.4, aplicată recursiv), taie ce nu contează (zgomot la denoising, biți la compresie, benzi la analiză) și reconstruiește cu filtrele de sinteză, exact sau controlat de aproape. Se schimbă doar ce înseamnă „nu contează”: zgomotul muscular la ECG (Capitolul 5), biții pe care ochiul nu-i vede la JPEG2000 (Capitolul 4), tot ce nu e vârf spectral la recunoașterea muzicii.

Localizarea simultană în timp și frecvență, motivul întregului Capitolul 3, e motivul concret pentru care toate aplicațiile de mai sus funcționează.

1

2

3

4

5

6

7

8

9

10

11

Un singur mecanism, cinci aplicații

Tot ce am văzut astăzi este același gest, repetat: descompune semnalul pe scări, taie ce nu contează, reconstruiește. Se schimbă doar ce înseamnă "nu contează".

Descompune aceeași bancă de filtre, aplicată recursiv pe aproximare (algoritmul lui Mallat)	Taie sub un prag: zgomot la denoising, biți la compresie, benzi la analiză	Reconstruiește filtrele de sinteză readuc semnalul, exact sau controlat de aproape
--	---	---

ECG tai zgomotul muscular, păstrezi complexul QRS	EEG separi benzile pe scări de frecvență	Shazam păstrezi doar vârfurile spectrale, restul e redundanță	Denoising tai coeficienții mici, semnalul rămâne în cei mari	JPEG2000 tai biții pe care ochiul nu-i vede, la același bitrate
--	---	--	---	--

De aceea localizarea simultană în timp și frecvență nu e o curiozitate matematică: e chiar motivul pentru care toate cele cinci funcționează.

↶

×

< Anterior

48 / 50

Următor >

Figura 9.1: Ecranul de sinteză al turului: descompune / taie / reconstruiește, aceleași trei verbe explicând ECG, EEG, recunoașterea muzicii, denoising și JPEG2000.

Capitolul 10

Cum e construit, verificat și publicat

Un ultim capitol, mai scurt, pentru cine vrea să știe și partea de inginerie din spatele aplicației, nu e conținut de curs, dar explică de ce demo-urile din capitolele anterioare pot fi luate ca atare.

10.1 Pe scurt, arhitectura

Front-end-ul e React peste Vite. Back-end-ul e FastAPI cu NumPy, SciPy și PyWavelets. Fiecare ecran e o intrare într-un singur tablou de date, nu un component separat, iar douăzeci și trei din cele cincizeci montează integral o componentă de demo interactiv, aceleași componente pe care bara laterală le poate deschide și individual, în afara turului. Toate desenează pe o suprafață fixă de 1440×810 pixeli virtuali, scalată proporțional la fereastra reală, ca un ecran gândit pentru un videoproiector să nu piardă nimic pe un monitor mai mic.

Calculul e împărțit în două, și distincția merită spusă limpede. Ecranele care raportează cifre măsurate trimit parametrii către back-end și primesc înapoi rezultatul unei biblioteci consacrate: filtrele, kernelele pe imagini reale, denoisingul, comparația la bitrate egal, electrocardiograma și extragerea benzilor EEG trec toate prin NumPy, SciPy și PyWavelets. Ecranele care ilustrează un mecanism calculează în browser, cu filtre Haar scrise direct în JavaScript: descompunerea pas cu pas, banca de filtre, piramida, reconstrucția, scalograma, cutiile Heisenberg și etapele intermediare ale pipeline-ului JPEG. Ele arată forma unei idei pe un semnal ales anume, nu o măsurătoare pe date reale, și de aceea nu poartă cifre pe care textul să se sprijine.

10.2 Verificare automată

Corectitudinea vizuală a celor cincizeci de ecrane se verifică prin trei unelte separate, fiecare acoperind o clasă diferită de defect, în loc de o singură trecere manuală.

Prima parcurge tot turul într-un browser real și verifică opt clase de problemă pe fiecare ecran (Tabelul 10.1), plus erorile de consolă JavaScript. Conform ultimei rulări complete, verificarea e curată pe toate cele cincizeci de ecrane, la două rezoluții diferite.

A doua unealtă merge mai departe decât o simplă citire: apasă efectiv fiecare buton, mișcă fiecare cursor, comută fiecare selector și bifează fiecare căsuță de pe fiecare ecran, cu o captură înainte și una după fiecare acțiune (Figura 10.1). Măsoară dacă vreun element s-a deplasat neașteptat pe ecran, verifică geometric dacă imaginile și canvas-urile rămân în interiorul cutiei lor și colectează orice eroare nouă de consolă. Ultima rulare completă a apăsă sau tras 911 controale, pe toate cele cincizeci de ecrane, fără niciunul lipsit de un nume accesibil și fără nicio eroare.

A treia unealtă scanează codul sursă, nu ecranul: caută orice caracter Unicode folosit în locul unei formule LaTeX sau al unei iconițe proprii de interfață, pe principiul că matematica se randează mereu prin LaTeX, iar orice iconiță e un desen propriu, nu un caracter de font care arată diferit de la un sistem la altul. E curată pe tot front-end-ul.

Niciuna dintre cele trei unelte nu vede ce se desenează în interiorul unui element de tip canvas, și acolo trăiesc majoritatea graficelor din tur: spectre, răspunsuri de filtru, forme de wavelet, scalograme. O suprapunere între o etichetă și o linie de grilă, acolo, nu apare în niciun raport automat. Compensarea

Tabela 10.1: Cele opt clase de defect verificate automat pe fiecare ecran.

Clasă	Ce detectează
<code>pageScroll</code> <code>outOfFrame</code>	Suprafața de design scrolează, conținutul depășește 1440×810 .
<code>innerScroll</code>	Un element vizibil (buton, input, imagine, canvas, text) iese din viewport, cu toleranță de 2 pixeli.
<code>overlaps</code>	Un element are conținut tăiat sau scrolabil intern (<code>scrollHeight</code> peste <code>clientHeight</code>).
<code>occludedControls</code>	Două elemente vizibile fără relație de descendență se suprapun cu cel puțin 6 pixeli pe ambele axe.
<code>smallMath</code>	Un control al cărui centru, testat la trei înălțimi, indică sub cursor un element diferit, prinde acoperirea de către un element exclus din <code>overlaps</code> , de exemplu un <code>canvas</code> .
<code>rawTokens</code> <code>duplicateTitles</code>	O formulă KaTeX randată sub pragul de lizibilitate (14px inline, 18px afișată, în pixeli de design).
	Un token <code>\$...\$</code> sau <code>[cite:...]</code> ajuns pe ecran ca text brut.
	Un ecran de tip <code>embed</code> își afișează propriul titlu, deasupra titlului din metadata turului.

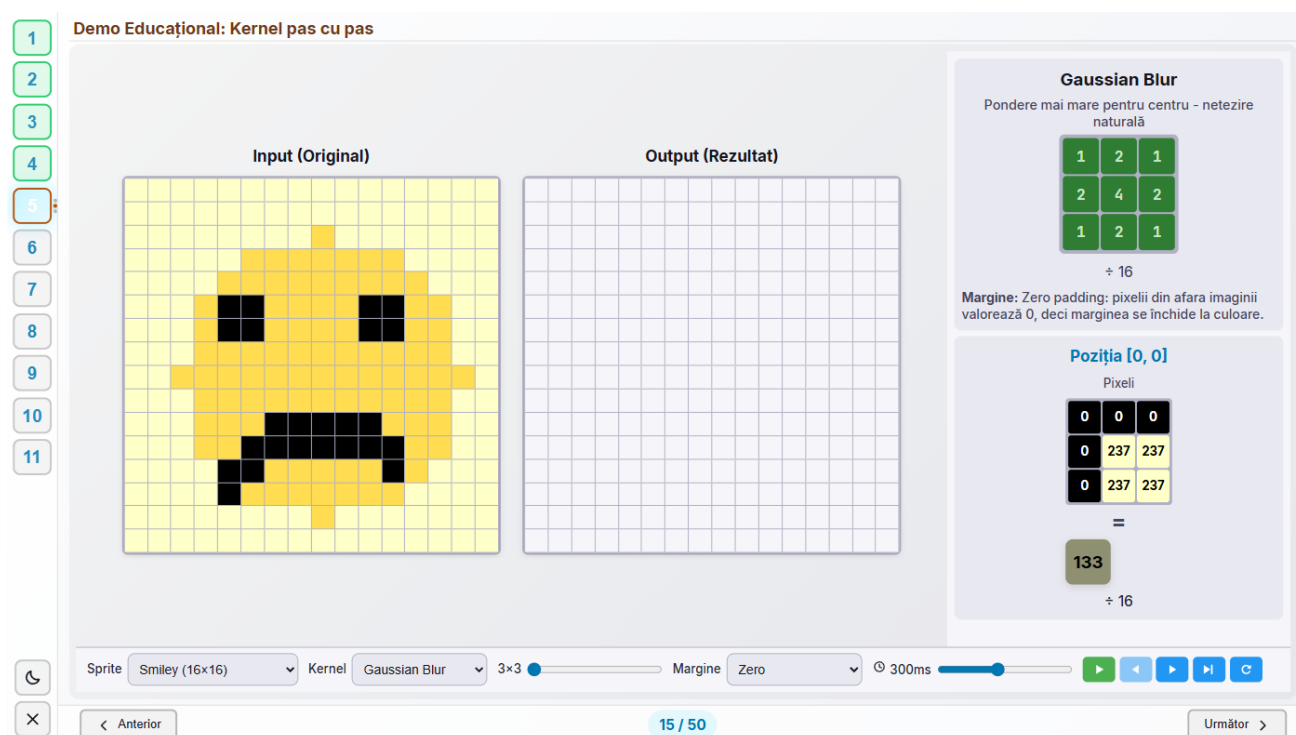


Figura 10.1: Un ecran tipic pentru trecerea de interacțiune: selector de sprite, selector de kernel, cursor de dimensiune, selector de margine, cursor de viteză și cinci butoane de transport, fiecare apăsător sau tras automat, cu o captură înainte și una după.

e directă: citirea capturilor de ecran salvate de aceste unelte, ecran cu ecran, exact așa au fost alese și cele douăzeci și două de figuri din acest document.

10.3 Rulare locală

Back-end-ul pornește din `backend/`, după instalarea dependențelor (FastAPI, Uvicorn, NumPy, SciPy, Pillow, PyWavelets, Pydantic). Front-end-ul pornește din `frontend/`, după `npm install`. Serverul de dezvoltare Vite redirectionează automat orice cerere către `/api` spre back-end, așa că front-end-ul n-are niciun URL scris în cod, nici măcar în dezvoltare. Fiecare ecran al turului se deschide direct, cu propriul *deep link*.

10.4 Publicarea pe server

Aplicația rulează dintr-o singură imagine Docker, construită în doi pași: front-end-ul se compilează întâi, apoi rezultatul e copiat peste back-end, care ajunge astfel să servească, pe același port, și pagina, și API-ul. Local, imaginea se construiește și pornește cu o singură comandă `docker compose`.

Publicarea la <https://ale0204.student-dev.ro/dsp/> nu construiește nimic pe mașina de acasă: sursa proiectului se împachetează și se trimite pe server, imaginea se construiește direct acolo, deci mașina de acasă n-are nevoie de Docker instalat sau pornit, doar trimite codul, iar la final containerul pornește în spatele reverse proxy-ului serverului, care termină conexiunea securizată și trimite mai departe cererile către aplicație. Containerul nu publică niciun port propriu.

Fișierele care descriu un server anume, rețeaua în care intră containerul, contul și directorul de pe gazdă, stau în afara depozitului public, pentru că depind de gazdă, nu de aplicație. Depozitul rămâne astfel rulabil de oricine, oriunde, fără nicio adresă scrisă în cod.

Bibliografie

Ordinea de mai jos e cea din registrul sursă al aplicației ([frontend/src/components/shared/sources.js](#)), ordinea primei apariții a fiecărei surse în tur, astfel încât numărul [n] din orice figură a acestui document coincide cu numărul citat de aplicație.

Bibliografie

- [1] Digital Cinema Initiatives. *Digital Cinema System Specification*, v1.4.4, secțiunea 4.2. DCI, LLC, 2024. <https://documents.dcmovies.com/DCSS/>
- [2] A. L. Wang. *An industrial-strength audio search algorithm*. Proc. ISMIR, 2003. <https://www.ee.columbia.edu/~dpwe/papers/Wang03-shazam.pdf>
- [3] R. G. Lyons. *Understanding Digital Signal Processing*, ediția a 3-a. Prentice Hall, 2011.
- [4] G. Strang, T. Nguyen. *Wavelets and Filter Banks*. Wellesley-Cambridge Press, 1996.
- [5] D. Esteban, C. Galand. *Application of quadrature mirror filters to split band voice coding schemes*. Proc. IEEE ICASSP, 1977. <https://doi.org/10.1109/ICASSP.1977.1170341>
- [6] M. J. T. Smith, T. P. Barnwell III. *A procedure for designing exact reconstruction filter banks for tree-structured subband coders*. Proc. IEEE ICASSP, 1984. <https://doi.org/10.1109/ICASSP.1984.1172486>
- [7] C. E. Shannon. *Communication in the presence of noise*. Proc. IRE 37(1), 10-21, 1949. <https://doi.org/10.1109/JRPROC.1949.232969>
- [8] I. Daubechies. *Ten Lectures on Wavelets*. SIAM, 1992. <https://doi.org/10.1137/1.9781611970104>
- [9] I. Daubechies. *Orthonormal bases of compactly supported wavelets*. Comm. Pure Appl. Math. 41(7), 1988. <https://doi.org/10.1002/cpa.3160410705>
- [10] D. S. Taubman, M. W. Marcellin. *JPEG2000: Image Compression Fundamentals, Standards and Practice*. Kluwer Academic, 2002.
- [11] W. Sweldens. *The lifting scheme: A construction of second generation wavelets*. SIAM J. Math. Anal. 29(2), 1998. <https://doi.org/10.1137/S0036141095289051>
- [12] S. Mallat. *A Wavelet Tour of Signal Processing: The Sparse Way*, ediția a 3-a. Academic Press, 2008. <https://www.sciencedirect.com/book/9780123743701/a-wavelet-tour-of-signal-processing>
- [13] S. Mallat. *A theory for multiresolution signal decomposition: the wavelet representation*. IEEE Trans. Pattern Anal. Mach. Intell. 11(7), 674-693, 1989. <https://doi.org/10.1109/34.192463>
- [14] M. V. Perez et al. (Apple Heart Study). *Large-scale assessment of a smartwatch to identify atrial fibrillation*. N. Engl. J. Med. 381(20), 1909-1917, 2019. <https://doi.org/10.1056/NEJMoa1901183>
- [15] J. P. Martínez, R. Almeida, S. Olmos, A. P. Rocha, P. Laguna. *A wavelet-based ECG delineator: evaluation on standard databases*. IEEE Trans. Biomed. Eng. 51(4), 570-581, 2004. <https://doi.org/10.1109/TBME.2003.821031>
- [16] D. L. Donoho, I. M. Johnstone. *Ideal spatial adaptation by wavelet shrinkage*. Biometrika 81(3), 425-455, 1994. <https://doi.org/10.1093/biomet/81.3.425>

- [17] ITU-T. *Rec. T.81: Digital compression and coding of continuous-tone still images*. ITU-T / ISO-IEC 10918-1, 1992. <https://www.w3.org/Graphics/JPEG/itu-t81.pdf>
- [18] Z. Wang, A. C. Bovik, H. R. Sheikh, E. P. Simoncelli. *Image quality assessment: from error visibility to structural similarity*. IEEE Trans. Image Process. 13(4), 600-612, 2004. <https://doi.org/10.1109/TIP.2003.819861>
- [19] A. V. Oppenheim, R. W. Schaffer. *Discrete-Time Signal Processing*, ediția a 3-a. Prentice Hall, 2010.